Rapport Projet Rust

GAMONET Hélias

May 15, 2026

Contents

1 Introduction **2**

2 Fonctionnalités **2**
2.1 Connexion sécurisée 2
2.2 Authentification 2
2.3 Multi-utilisateurs 2
2.4 Commandes 3

3 Instructions d'utilisation **3**
3.1 Serveur 3
3.2 Client 3

4 Choix de conception **4**
4.1 Architecture 4
4.2 Connexion 4
4.3 Gestion des messages 5
4.4 Gestion des utilisateurs 5
4.5 Multi-clients 5
4.6 Gestion des commandes 6

1 Introduction

Mon projet se nomme bsh (BlibShell) et propose l'utilisation d'un shell à distance avec communications sécurisées grâce à TLS. Deux binaires, client et serveur, permettent le fonctionnement du logiciel. Le serveur est capable de gérer une quantité non-limitée de client et propose une gestion des utilisateurs au dessus des utilisateurs Linux. Le programme ne fonctionne que sur Linux car il repose sur son système de fichiers et sa gestion des utilisateurs.

2 Fonctionnalités

2.1 Connexion sécurisée

La connexion entre serveur et client est sécurisée grâce à TLS. Le client et le serveur possèdent un certificat à clef publique et privée signé par la même authorité de certification. Les deux parties s'authentifient mutuellement. Les certificats utilisés sont des certificats X509 v3 contenant des clés RSA-2048. Ils sont signés par l'authorité de certification.

2.2 Authentification

Après la mise en place de la connexion TLS, le client doit prouver qu'il a le droit d'utiliser l'utilisateur qu'il demande. Pour cela deux méthodes s'offrent à lui : par mot de passe ou par certificat. Le client choisi la méthode d'authentification avant la connexion au serveur.

La connexion par mot de passe est simple. D'abord le client tape son mot de passe qui est envoyé, via la connexion TLS, au serveur. Le serveur produit un hash du mot de passe grâce à Argon2 et vérifie ce hash avec celui qu'il possède pour l'utilisateur demandé. Si les deux sont égaux alors le client est authentifié, sinon le client est déconnecté.

L'authentification par certificat nécessite la connexion au préalable d'un client avec l'utilisateur demandé pour la configurer. D'abord le client demande au serveur d'initier l'authentification par certificat. Ensuite, le serveur produit un nonce qu'il envoie au client. Le client signe ce nonce avec sa clé privée et renvoie la signature au serveur. Enfin, le serveur vérifie la signature avec la clé publique correspondant à l'utilisateur demandé. Si la signature correspond alors le client est authentifié (il possède la clé privée qui correspond au certificat que le serveur possède pour cet utilisateur). Si la signature ne correspond pas, alors le client est déconnecté.

2.3 Multi-utilisateurs

Les utilisateurs mentionnés précédemment ne sont pas des utilisateurs Linux mais une abstraction de l'application. Bsh propose un système d'utilisateur mappé à des utilisateurs Linux. Le serveur possède un fichier ".user-X" et un fichier ".role-X" par utilisateur. Dans le fichier ".user-X", le serveur garde son identifiant, le hash de son mot de passe, et optionnellement un certificat utilisé pour l'authentification par certificat. Le fichier ".role-X" contient l'utilisateur Linux associé à l'utilisateur bsh, ainsi que son role (User ou Admin).

Un utilisateur au role User peut utiliser le shell normalement, changer son mot de passe et uploader son certificat. Un utilisateur au rol Admin à, en plus des permissions du role User, un accès aux commandes administrateurs du serveur. Ces commandes permettent d'ajouter des utilisateurs, en supprimer, et de les lister.

L'utilisateur Linux associé est utilisé comme UID pour le shell du client.

2.4 Commandes

Une fois connecté et authentifié, le client a accès à un shell. Les commandes qu'il tape dans son terminal sont envoyées au serveur qui les transmet directement au shell. Le stdout et stderr du shell sont transmis au client. Des commandes "builtins" sont disponibles. Elles correspondent en grande partie aux commandes administrateurs et utilisateurs. Une commande builtin est toujours précédée d'un "!".

3 Instructions d'utilisation

- cargo test - lance les tests disponibles.
- cargo build –release - compile le serveur et le client dans le fichier target. Tout est compris dans les deux binaires server et client.

Les certificats étant déjà inclus dans les binaires (voir 4.2), il n'y a pas besoin d'en générer.

3.1 Serveur

Le serveur nécessite d'être exécuté en tant que root, principalement pour protéger ses fichiers. En effet, les fichiers ".role-X" contiennent l'utilisateur Linux à utiliser pour un utilisateur bsh, si ce fichier était modifié il pourrait donner les droits root à un utilisateur lambda.

Avant la première utilisation, le serveur doit être lancé avec l'option "–setup". Cette option permet la configuration d'un premier utilisateur "admin" et créée l'arborescence de fichier nécessaire au fonctionnement du serveur.

Options :

- –setup - Exécute le serveur en mode setup.
- -p<port>, –port <port> - Défini le port sur lequel le serveur écoute.

- sudo ./server
- sudo ./server –setup
- sudo ./server -p 25000

3.2 Client

Le client ne nécessite aucune configuration préalable.

- ./client <user> <address:port>
- ./client admin 127.0.0.1:45000
- ./client admin 127.0.0.1:25000 -a certificate

Une seule option est disponible : -a, –auth <type>. Cette option permet de sélectionner la méthode d'authentification parmis les deux disponibles: c ou certificate et p ou password (p = défaut).

<user> correspond à l'utilisateur bsh à utiliser.

Une fois connecté et authentifié, le client a à sa disposition plusieurs commandes builtins :

- !exit - Se déconnecte.
- !user - Affiche les commandes utilisateurs (!change_password et !upload_publickey).
- !change_password - Ouvre un formulaire pour changer son mot de passe.
- !upload_publickey - Upload le certificat du client pour l'authentification par certificat.
- !admin - Affiche les commandes administrateurs (list_users, !add_user et !remove_user).
- !list_users - Liste tous les utilisateurs.
- !add_user <username> - Ouvre un formulaire pour ajouter un nouvel utilisateur.
- !remove_user <username> - Supprime l'utilisateur.

Il suffit ensuite de suivre les indications dans le terminal.

4 Choix de conception

4.1 Architecture

J'ai choisi de créer un workspace rust pour contenir mes 3 projets : server, client et common.

Server et client contiennent chacun leur implémentation, tandis que common contient les structures et fonctions communes aux deux projets : les messages, le receveur, etc...

4.2 Connexion

Pour gérer la connexion j'ai choisi la crate rustls. Grâce à son wrapper de TcpStream je suis capable d'utiliser TLS très facilement. Cette crate semble être le choix standard pour TLS sous rust.

J'utilise également tokio-rustls pour intégrer rustls à tokio que j'utilise comme runtime asynchrone.

Enfin, j'utilise rustls-pemfile pour charger les certificats spécifiquement pour TLS.

Pour les certificats dans les autres contextes (signature de nonce et vérification, upload de certificat, ...), j'utilise openssl. Les certificats sont inclus dans les binaires grâce à include_bytes!. Cela me permet de prégénérer des certificats signés par la même authorité de certification : inclure une authorité de certifications dans tous les binaires n'aurait pas de sens. La bonne chose à faire serait d'avoir un serveur capable de signer des certificats mais cela demanderait bien plus de travail et serait en dehors du scope du projet.

4.3 Gestion des messages

Un message est soit une requête, soit une réponse. Le choix de ces termes n'est pas forcément idéal car une requête correspond à un message provenant du client, alors qu'une réponse est un message provenant du serveur.

Les messages sont définis dans deux enum Request et Response. Ces deux enums utilisent beaucoup les "struct variants", ce qui me permet de placer des contenus différent par type de message tout en gardant tout dans un seul type.

Pour sérialiser les messages, j'utilise serde. Les messages ne sont pas sérialisés en JSON, mais en MessagePack qui est une alternative au JSON tout en étant plus rapide et légé. J'ai également fait le choix du protocole de sérialisation par rapport au support de serde. J'utilise donc rmp, rmp-serde et rmpv.

Les enums de message implémentent les traits serde::Serialize et serde::Deserialize.

La réception et l'envoi de messages sont séparés en deux taches distinct. Dès le départ je voulais éviter le schéma "envoi de message, attente d'une réponse, répète". Mon implémentation en 2 taches avec un channel mpsc me permet de faire les deux simultanément et d'implémenter des nouveux types de messages assez facilement. Pour que cela fonctionne correctement, je dois faire en sorte que la gestion de chaque message soit indépendante. Par exemple, pour gérer un AuthChallenge, je vérifie d'abord si le client est dans le bon état, puis j'exécute la logique correspondante au message. Un problème est à noter, je ne peux pas implémenter de "suite de messages" ; Actuellement, un client peut tout à fait écrire alors qu'il attend l'output de sa commande.

La tâche de réception de messages est commune au serveur et au client. Logiquement, j'ai créé une structure générique qui permet cela. Les enums Response et Request implémentent le trait Message<T>.

4.4 Gestion des utilisateurs

J'ai fait le choix de gérer mes propres utilisateurs au dessus des utilisateurs Linux. Cela me permet de gérer les mots de passe, les certificats, les rôles, etc... Chaque utilisateur bsh possède un mapping vers un utilisateur Linux qui permet de donner des permissions différentes.

Les données des utilisateurs sont stockées dans deux dossiers : users et roles. Chaque utilisateur est représenté par un fichier ".user-X" et ".role-X". Cette séparation vient du fait qu'un client peut modifier son mot de passe ou uploaded un certificat, il a donc un certain contrôle sur le fichier ".user-X". Le fichier ".role-X" contient le mapping vers l'utilisateur Linux et le rôle de l'utilisateur. Laisser un client non-autorisé modifier ces informations serait dramatique, c'est pourquoi je sépare les deux.

Les données utilisateur sont sérialisées en JSON avec serde.

4.5 Multi-clients

Le serveur n'impose pas de plafond fixe vis à vis du nombre de clients connectés. A la connexion TCP, un client se voit affecté une tâche tokio qui va la gérer. Il possède également une tâche pour la réception des messages comme expliqué précédemment. Un process bash est aussi exécuté. Une structure client centralise les informations d'un client : SocketAddress, process shell avec ses stdin, stdout et stderr, et l'utilisateur qu'il utilise.

Les fonctions de gestion de messages prennent la structure Client en argument.

Grâce à tout cela, la fonction main écoute le flux TCP et spawn une tâche à chaque connexion. Lorsque le client se déconnecte, la tâche en question est autonome et arrête tout ce qui est en lien avec le client. De cette manière, la gestion de plusieurs clients en simultané est assez évidente.

4.6 Gestion des commandes

Les commandes sont envoyées ligne par ligne. Lorsque le client ajoute une ligne dans son terminal, elle est envoyée au serveur qui la donne tel quel dans le stdin du shell associé au client. Le serveur écoute le stdout et stderr du shell et envoi leur contenu au client. Tout est asynchrone, le serveur écoute en continue le stdout et stderr.

De cette manière, le serveur peut gérer les commandes comme n'importe quel autre message. Le seul problème étant que je ne peux pas trop gérer les processus intéractif comme Vim.

Le client gère un shell complet, il peut donc définir des variables d'environnement, naviguer dans l'arborescence de fichiers et faire tout ce qui est possible de faire dans un shell classique.